Python Programming – Application-Based Machine Test

Assessment – Set 3

Duration: 90 Minutes
Total Questions: 3
Maximum Marks: 100
Programming Language: Python 3.x
Assessment Type: Machine Test
Difficulty: Easy → Medium → Hard

Rules and Regulations

1. The total duration of the assessment is **90 minutes**.
2. Students must attempt the assessment using **Python 3.x**.
3. All programs must be executed and tested before submission.
4. Students must follow the **input and output format** specified in each question.
5. Programs should work for **all valid inputs**, not only the sample inputs provided.
6. **Hard-coding** the expected output for the given sample input is strictly prohibited.
7. Students may use Google colab.
8. Students may use Python's **built-in functions and standard language features**.
9. Students must not use:
    - Internet search
    - ChatGPT or other AI tools
    - Copied code from websites
    - Code generated by external tools
    - Communication with other students
10. Students may refer only to the **Python environment/documentation explicitly permitted by the trainer**.
11. Each student must work **independently**.
12. Students must save their programs using the prescribed file names.
13. Students should test their programs with different inputs, including boundary cases.
14. The evaluator may execute the submitted program with **hidden test cases**.
15. Marks will be awarded based on **correctness, logic, implementation, handling of edge cases, and code quality**.
16. A program that produces the correct output only for the sample input will **not receive full marks**.

17. Students should manage their time appropriately. The recommended time allocation is:
- Q1 – 20 minutes
- Q2 – 30 minutes
- Q3 – 40 minutes

18. Any form of malpractice, collaboration, or unauthorized assistance will result in **disqualification from the assessment**.

Question 1 – Easy

Parking Fee Calculator

A shopping mall wants to automate its parking fee calculation.

The parking fee depends on the type of vehicle and the number of hours parked.

Vehicle Types

Vehicle	First 2 Hours	Every Additional Hour
Car	₹50	₹20
Bike	₹30	₹10
SUV	₹70	₹30

For example, if a **Car** is parked for 5 hours:

- First 2 hours = ₹50
- Remaining 3 hours = 3 × ₹20
- Total = ₹110

Additional Rules

- Parking for exactly 2 hours is charged only at the first-2-hours rate.
- Parking for less than 1 hour is considered **1 hour**.
- If the vehicle type is not Car, Bike, or SUV, display:

Invalid vehicle type

- If the number of hours is zero or negative, display:

Invalid parking duration

Input Format

Vehicle_Type
Hours

Sample Input

Car
5

Sample Output

Vehicle: Car
Hours: 5
Parking Fee: 110

Sample Input 2

Bike
2

Sample Output 2

Vehicle: Bike
Hours: 2
Parking Fee: 30

Suggested Time

20 minutes

Marks

20

Question 2 – Medium

Movie Theatre Seat Booking Analyzer

A movie theatre has multiple shows. Each show has a fixed number of seats.

For each show, the following information is provided:

- Show name
- Total seats
- Number of booked seats

The theatre wants to analyze the occupancy of each show.

Occupancy Rules

Calculate:

Occupancy Percentage = (Booked Seats / Total Seats) × 100

Classify the show as:

Occupancy	Category
80% or above	House Full
60%–79%	High Demand
40%–59%	Moderate
Below 40%	Low Demand

Your Task

Write a Python program that accepts information for N shows and displays:

1. Show name
2. Available seats
3. Occupancy percentage
4. Occupancy category

At the end, display:

- Show with the highest occupancy
- Total number of booked seats
- Total number of available seats
- Number of shows classified as **High Demand** or **House Full**

Input Format

The first line contains:

N

The next N lines contain:

Show_Name Total_Seats Booked_Seats

Sample Input

4
MorningShow 100 35
AfternoonShow 120 75
EveningShow 150 135
NightShow 100 55

Sample Output

MorningShow - Available: 65 - Occupancy: 35.00% - Low Demand
AfternoonShow - Available: 45 - Occupancy: 62.50% - High Demand
EveningShow - Available: 15 - Occupancy: 90.00% - House Full
NightShow - Available: 45 - Occupancy: 55.00% - Moderate

Highest Occupancy: EveningShow
Total Booked Seats: 300
Total Available Seats: 170
High Demand/House Full Shows: 2

Additional Requirements

The program should correctly handle:

- 0 booked seats
- All seats booked
- Exactly 40% occupancy
- Exactly 60% occupancy
- Exactly 80% occupancy

Suggested Time

30 minutes

Marks

30

Question 3 – Hard

Employee Leave Management System

A company wants to develop a simple employee leave management system.

Create a Python class called Employee with the following attributes:

- employee_id
- employee_name
- department
- leave_balance

Each employee initially has a certain number of available leave days.

Create appropriate methods to manage leave requests.

Method 1 – apply_leave(days)

An employee can request leave only when:

- days is greater than 0
- The requested number of days does not exceed the available leave balance

If sufficient leave is available:

- Deduct the requested days from the leave balance.
- Display:

Leave Approved

Otherwise display:

Insufficient Leave Balance

For an invalid number of days, display:

Invalid Leave Request

Method 2 – cancel_leave(days)

If an employee cancels previously taken leave:

- Add the specified number of days back to the leave balance.
- The number of days must be greater than 0.

If invalid:

Invalid Cancellation

Method 3 – check_balance()

Display the employee's current leave balance.

Method 4 – display_employee()

Display:

- Employee ID
- Employee Name
- Department
- Current Leave Balance

Input Format

The first line contains:

Employee_ID Employee_Name Department Initial_Leave_Balance

The second line contains the number of operations.

The following lines contain one of:

APPLY days
CANCEL days
BALANCE
DISPLAY

Sample Input

E101 Anu IT 20
6
APPLY 5
BALANCE
APPLY 10
CANCEL 3
APPLY 8
DISPLAY

Sample Output

Leave Approved
Leave Balance: 15
Leave Approved
Leave Balance: 8
Leave Approved
Employee ID: E101
Employee Name: Anu
Department: IT
Leave Balance: 0

Additional Requirements

The program should correctly handle:

- Applying exactly the available balance
- Applying more leave than available
- Applying zero days
- Applying negative days
- Cancelling leave
- Multiple consecutive leave requests
- Multiple operations on the same employee

The program should use **classes and objects** to implement the employee leave management functionality.

Suggested Time

40 minutes

Marks

50